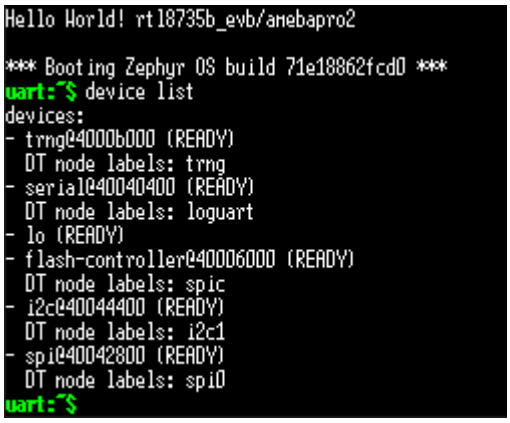
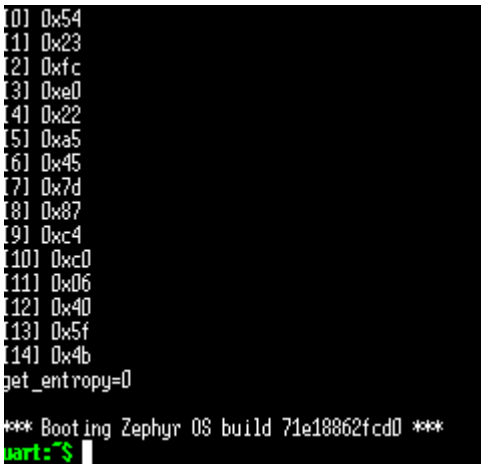
- Log UART driver

1. Build sample: zephyr/samples/hello_world/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and the Hello World! rtl8735b_evb/amebapro2 log is shown.
4. Input device list command to shell, and shell can respond to show device list.



- Entropy driver

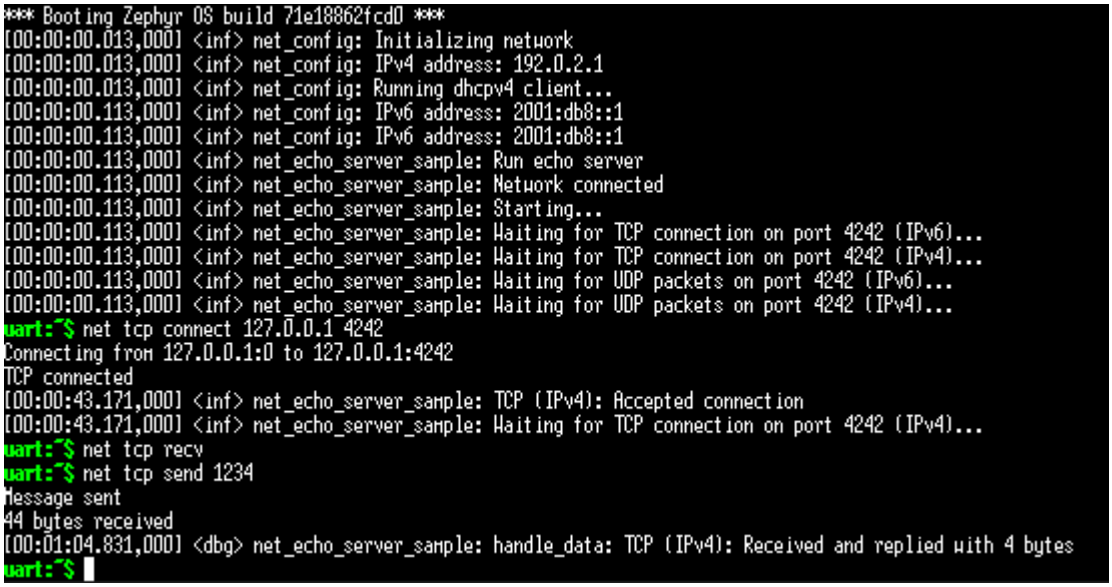
1. Build sample: platform/samples/entropy/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and a random value of 15 bytes is shown.



- Networking

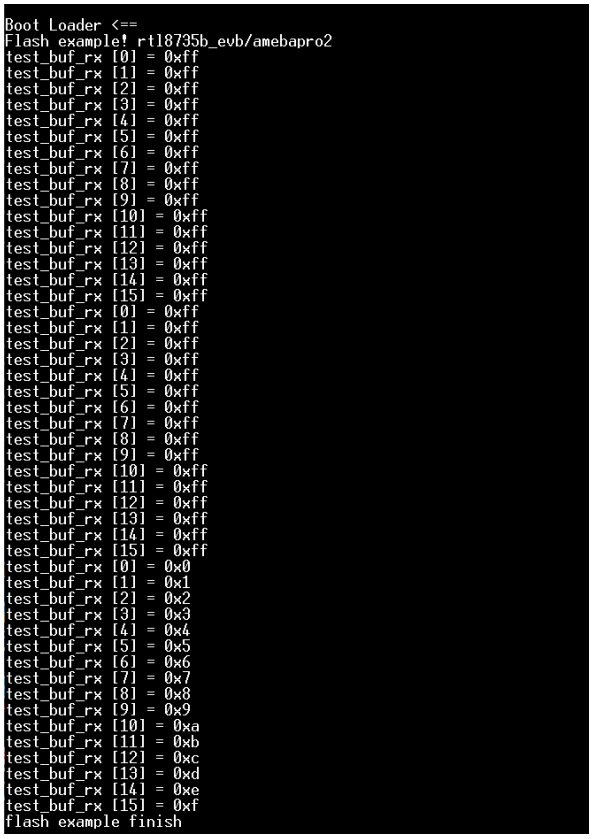
1. Build sample: zephyr/samples/net/sockets/echo_server/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and echo server is started to wait for TCP connection.
4. Input net tcp connect 127.0.0.1 4242 command to shell, and shell TCP client will connect to the local echo server.
5. Input net tcp recv command to shell, shell TCP client will be able to handle the received data.

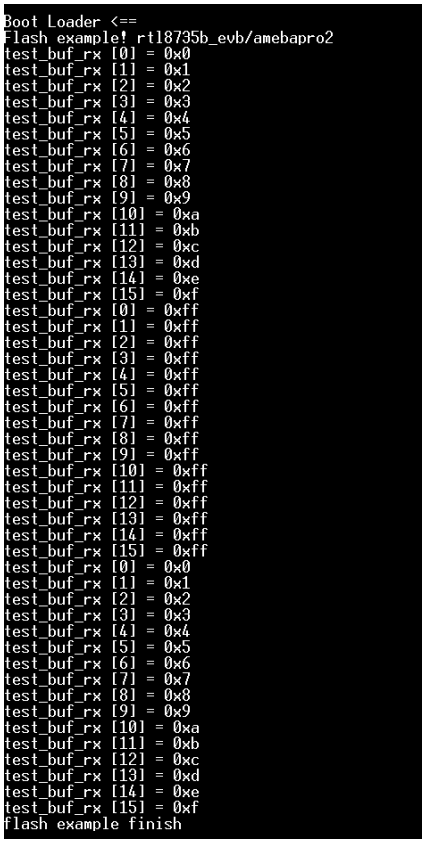
6. Input net tcp send 1234 command to shell, shell TCP client will send 4 bytes TCP data to echo server. Shell TCP client will receive a packet of 44 bytes from echo server.



- Flash driver

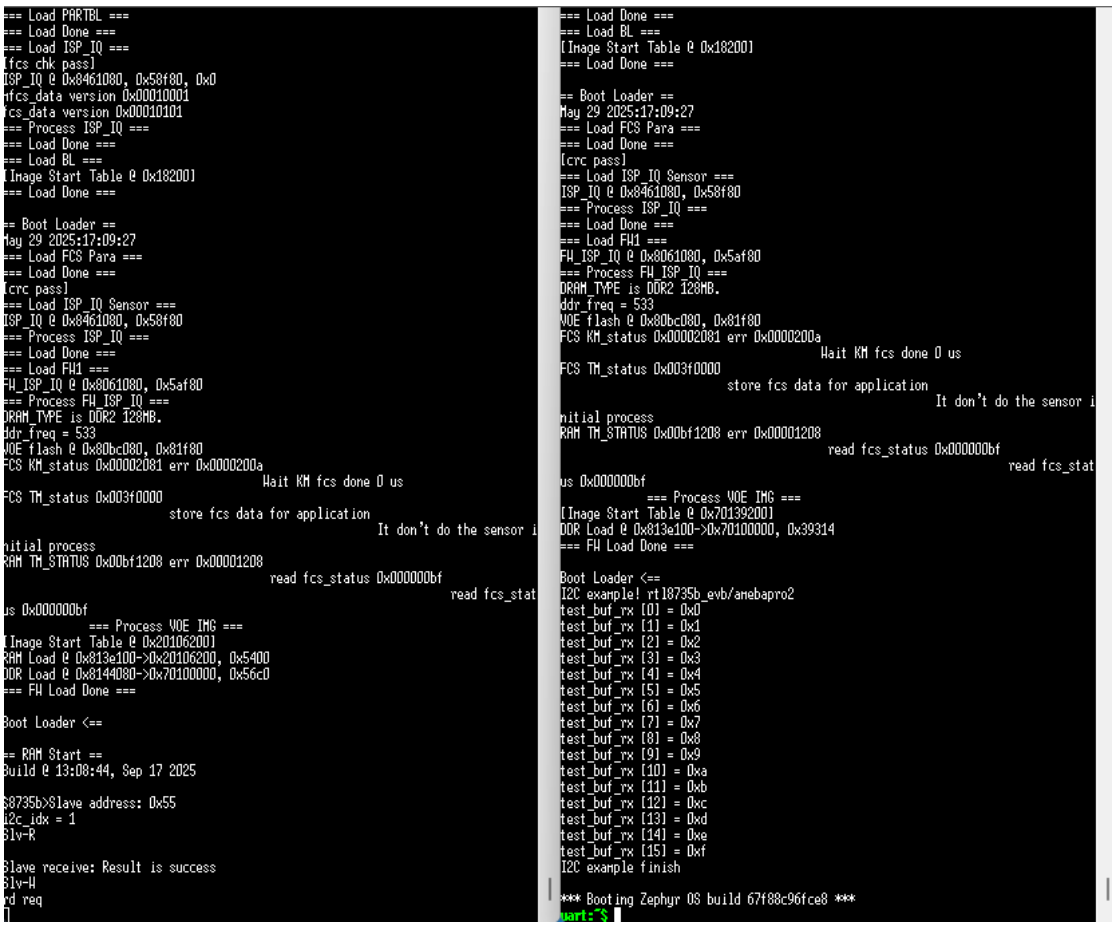
1. Build sample: platform/samples/flash/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and the sample will write data to flash. The log will show the read data from flash after flash write.
4. The previously written data can be read when the next reset.





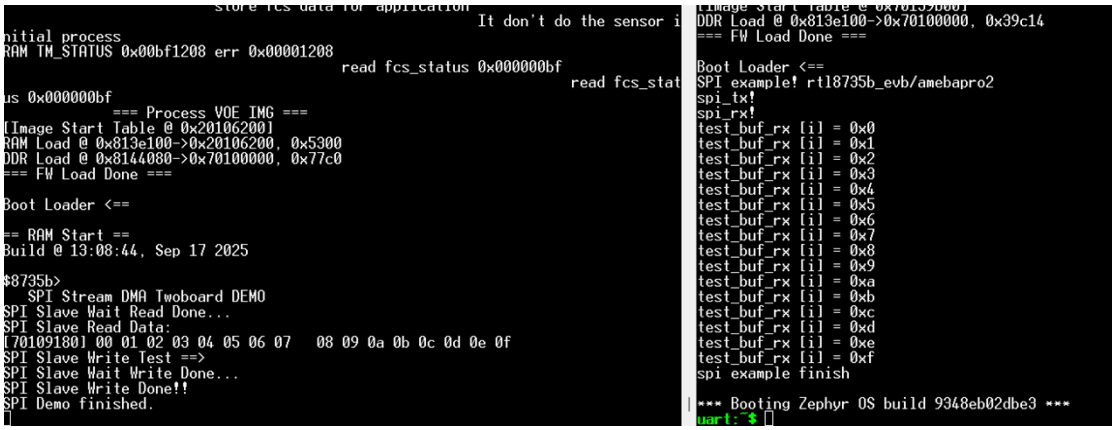
- I2C master driver

1. Build sample: platform/samples/i2c/
2. Use PG tool to download flash_zephyr.bin to master EVB.
3. Use PG tool to download flash_ntz_i2c.bin to slave EVB.
4. Master SCL GPIOF_1 is connected to slave SCL GPIOF_1, and an external pull-up resistor (4.7k) is required.
5. Master SDA GPIOF_2 is connected to slave SDA GPIOF_2, and an external pull-up resistor (4.7k) is required.
6. Reset slave EVB first and then reset master EVB. The slave receives the data from the master and then sends the data back to the master.
7. The log of master EVB (the log on the right) will show the received data from slave EVB.



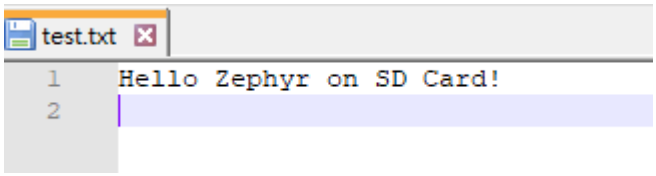
- SPI master driver

1. Build sample: platform/samples/spi/
2. Use PG tool to download flash_zephyr.bin to master EVB.
3. Use PG tool to download flash_ntz_spi.bin to slave EVB.
4. Master SCLK GPIOE_1 is connected to slave SCLK GPIOE_1.
5. Master MISO GPIOE_2 is connected to slave MISO GPIOE_2.
6. Master MOSI GPIOE_3 is connected to slave MOSI GPIOE_3.
7. Master CS GPIOE_4 is connected to slave CS GPIOE_4.
8. Reset slave EVB first and then reset master EVB. The slave receives the data from the master and then sends the data back to the master.
9. The log of master EVB (the log on the right) will show the received data from slave EVB.



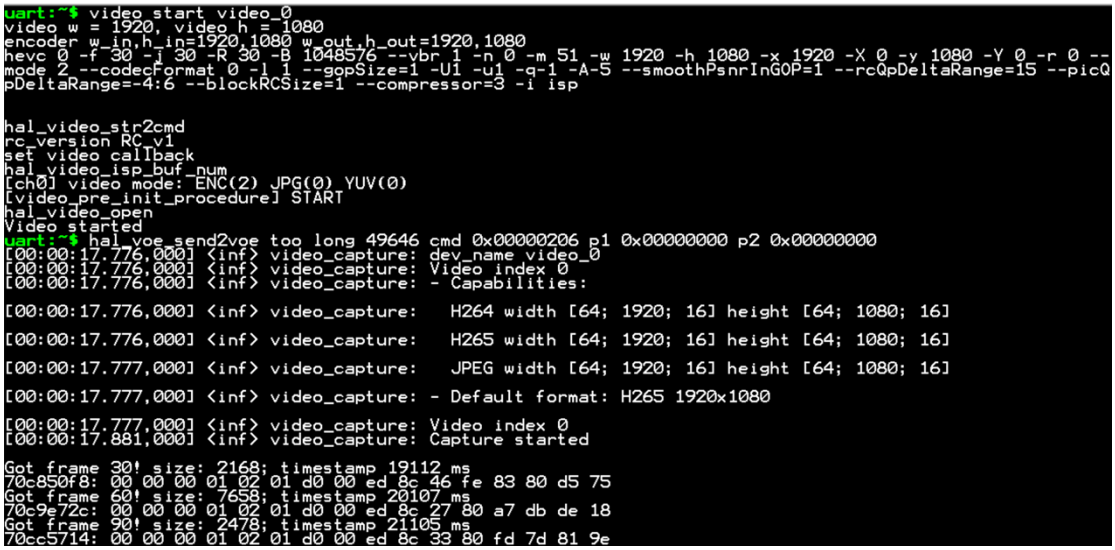
- SD Card driver

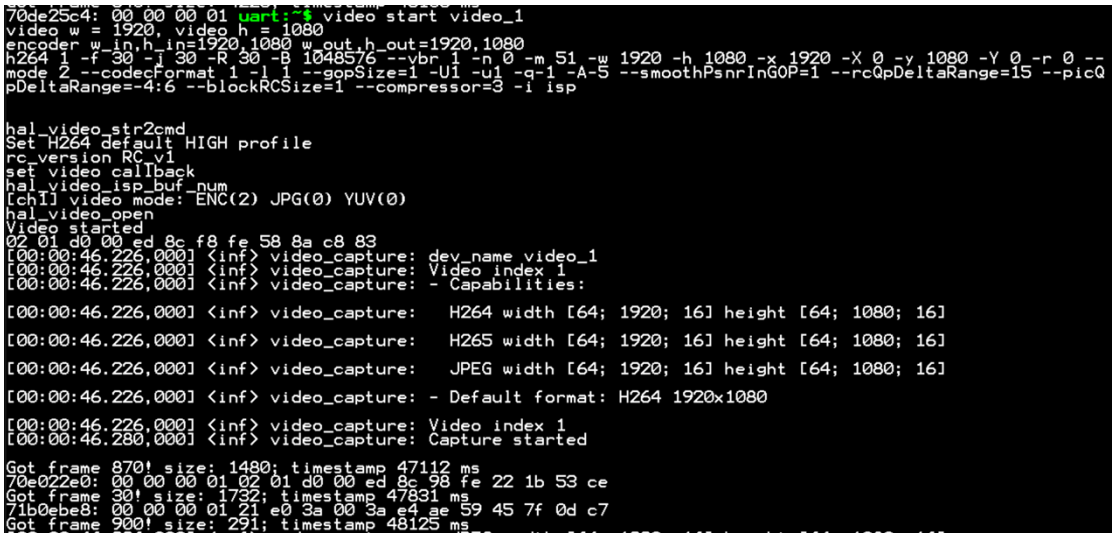
1. Build sample: platform/samples/sdcard/
2. Use PG tool to download flash_zephyr.bin.
3. Insert an SD card formatted with FAT32 or exFAT into your EVB board.
4. Reset the EVB, this sample will create a file named test.txt on SD card and write data to this file.
5. Afterward, you can insert the SD card into a PC to verify its contents.



- Video driver

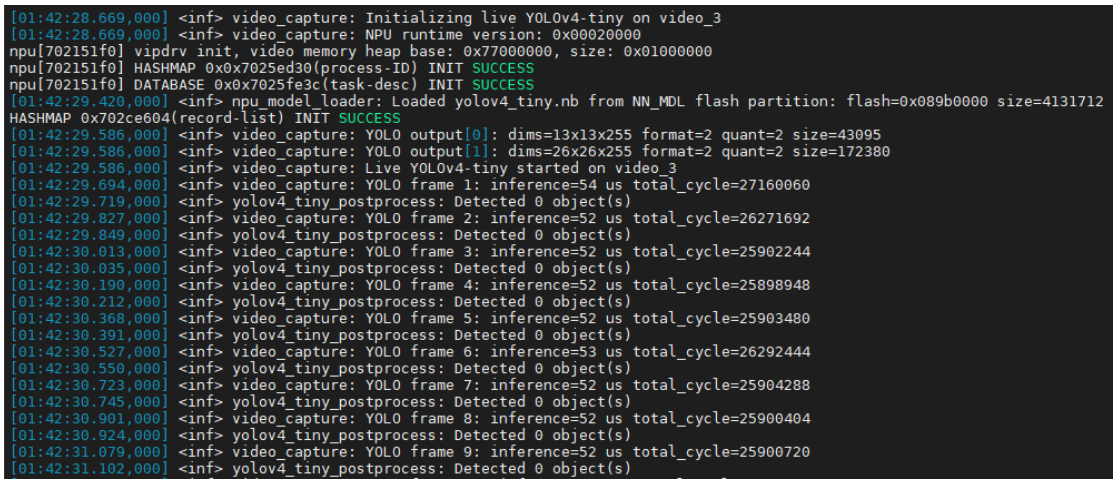
1. Build sample: platform/samples/video/
2. Use PG tool to download flash_zephyr.bin.
3. Connect GC2053 image sensor to EVB.
4. Reset the EVB, and input video start video_0 command to shell to start the channel video_0, and input video start video_1 command to shell to start the channel video_1. You should see video logs and frame data. Frame content will be displayed approximately every 30 frames. The video format for channel video_0 is HEVC and for channel video_1 is h264.





5. Input video start video_2 command to shell to start channel video_2. The video format for channel video_2 is NV12 (320x180 @ 10fps).
6. Motion detection is supported on video_2 only. Use the following commands:
   a. video md start video_2 — Start motion detection on video_2.
   b. video md stop video_2 — Stop motion detection.
   c. video md status video_2 — Show current motion detection status.
   d. video md sensitivity <0-100> video_2 — Set sensitivity level (0 = lowest, 100 = highest).
7. YOLOv4-tiny object detection is supported on video_3 only. The video format for channel video_3 is RGB888 planar (416x416 @ 5fps). Video frames are sent directly to the NPU as model input, and the detection results and inference performance are printed to the console. Use the following commands:
   a. video nn start — Start YOLOv4-tiny object detection. This command automatically starts video_3 if it is not already running and loads yolov4_tiny.nb from the NN model partition.

b. video nn stop — Stop object detection and unload the NPU model. Channel video_3 remains running.
c. video nn status — Show the current NN status, video_3 channel status, model information, completed inference frame count, error count, and last error.
d. video stop video_3 — Stop channel video_3. If object detection is running, the NPU model and related resources are also released automatically.



- GPIO driver

1. Build sample: platform/samples/gpio/
2. Use PG tool to download flash_zephyr.bin.
3. GPIOA_2 is aon GPIO irq input.
4. GPIOA_3 is aon GPIO output.
5. GPIOF_0 is pon GPIO irq input.
6. GPIOF_8 is pon GPIO output.
7. GPIOE_5 is syson GPIO irq input.
8. GPIOE_6 is syson GPIO output.
9. Reset the EVB. It can see the log and check the content.
10. The output pins will be set high for 3 seconds and then set low. Use a voltmeter to measure the voltage of the pins.
11. The input pins rising edge triggers the irq handler. Use a jumper wire to connect GND to 3.3V to create a GPIO rising edge.

```
nitial process
RAM TM_STATUS 0x00bf1208 err 0x00001208
                                        read fcs_status 0x000000bf
                                                                read fcs_stat
us 0x000000bf
            === Process VOE IMG ===
[Image Start Table @ 0x70139e00]
DDR Load @ 0x813e100->0x70100000, 0x39f14
=== FW Load Done ===

Boot Loader <==
gpio example! rtl8735b_evb/amebapro2
gpio_aon_get_raw 0x1

*** Booting Zephyr OS build 9348eb02dbe3 ***
uart:~$ gpio_pon_get 0x1
gpio_get 0x1
gpio_pon_ameba_isr
gpio_pon_ameba_isr
gpio_pon_ameba_isr
gpio_pon_ameba_isr
```

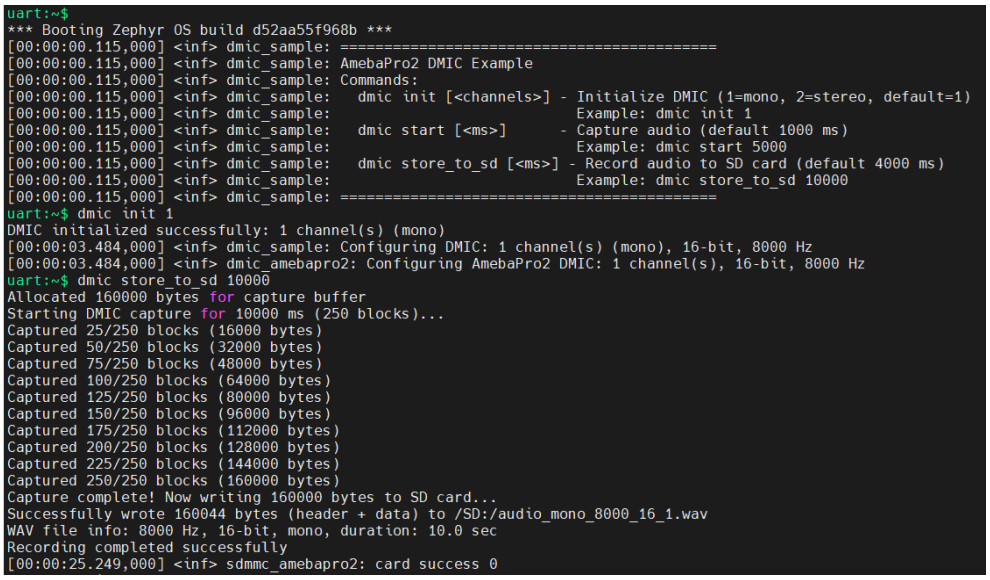
- Digital microphone (DMIC) driver

1. Build sample: platform/samples/audio/
2. Use PG tool to download flash_zephyr.bin.
3. Connect PDM Microphone Unit (SPM1423) to EVB.
   (e.g. connecting DMIC VDD to EVB V33, DMIC GND to EVB GND, DMIC DATA to EVB GPIOD18 and DMIC CLK to EVB GPIOD16)
4. Reset the EVB, and input dmic init 1 command to shell to initialize DMIC with mono mode, or input dmic init 2 command to initialize with stereo mode.
5. Start dmic recording
   a. Input dmic start 5000 command to capture 5 seconds of audio. You should see audio logs and statistics. Audio stats (RMS, volume, avg, min, max) will be displayed approximately every 1 second.

```
*** Booting Zephyr OS build d52aa55f968b ***
[00:00:00.115,000] <inf> dmic_sample: ========================================
[00:00:00.115,000] <inf> dmic_sample: AmebaPro2 DMIC Example
[00:00:00.115,000] <inf> dmic_sample: Commands:
[00:00:00.115,000] <inf> dmic_sample:   dmic init [<channels>] - Initialize DMIC (1=mono, 2=stereo, default=1)
[00:00:00.115,000] <inf> dmic_sample:                            Example: dmic init 1
[00:00:00.115,000] <inf> dmic_sample:   dmic start [<ms>]      - Capture audio (default 1000 ms)
[00:00:00.115,000] <inf> dmic_sample:                            Example: dmic start 5000
[00:00:00.115,000] <inf> dmic_sample:   dmic store_to_sd [<ms>] - Record audio to SD card (default 4000 ms)
[00:00:00.115,000] <inf> dmic_sample:                            Example: dmic store_to_sd 10000
[00:00:00.115,000] <inf> dmic_sample: ========================================
uart:~$ dmic init 1
DMIC initialized successfully: 1 channel(s) (mono)
[00:00:04.031,000] <inf> dmic_sample: Configuring DMIC: 1 channel(s) (mono), 16-bit, 8000 Hz
[00:00:04.031,000] <inf> dmic_amebapro2: Configuring AmebaPro2 DMIC: 1 channel(s), 16-bit, 8000 Hz
uart:~$ dmic start 10000
Capturing 10000 ms of audio...
Audio stats: rms=487, vol=1%, avg=-23, min=-1334, max=1143
Audio stats: rms=454, vol=1%, avg=-1, min=-1342, max=1554
Audio stats: rms=469, vol=1%, avg=-26, min=-1338, max=1833
Audio stats: rms=645, vol=1%, avg=-17, min=-2161, max=2263
Audio stats: rms=416, vol=1%, avg=-8, min=-1467, max=1113
Audio stats: rms=428, vol=1%, avg=-4, min=-1094, max=1288
Audio stats: rms=444, vol=1%, avg=8, min=-1341, max=1139
Audio stats: rms=1395, vol=4%, avg=12, min=-2813, max=2798
Audio stats: rms=3864, vol=11%, avg=5, min=-6534, max=6122
Audio stats: rms=14612, vol=44%, avg=-5, min=-21701, max=21660
DMIC capture completed
[00:00:14.065,000] <inf> dmic_sample: Starting DMIC capture for 10000 ms (250 blocks)...
[00:00:24.175,000] <inf> dmic_sample: Successfully captured 250 blocks
[00:00:24.175,000] <inf> dmic_sample: DMIC stopped
```

   b. Or input dmic store_to_sd 10000 command to record 10 seconds of audio to SD card as a WAV file. The audio format is 16-bit PCM at 8 kHz sample

rate.



- PWM driver

1. Build sample: platform/samples/pwm/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and the pwm example! rtl8735b_evb/amebapro2 log is shown.
4. GPIOF_6 is pwm channel 0, 20kHz duty 20%.
5. GPIOF_7 is pwm channel 1, 20kHz duty 40%.
6. GPIOF_8 is pwm channel 2, 20kHz duty 60%.
7. GPIOF_14 is pwm channel 8, 20kHz duty 80%.
8. GPIOF_15 is pwm channel 9, 20kHz duty 100%.
9. Observe the signal by using an oscilloscope or logic analyzer.

- ADC driver

1. Build sample: platform/samples/adc/
2. Use PG tool to download flash_zephyr.bin.
3. Reset the EVB, and the adc example! rtl8735b_evb/amebapro2 log is shown. And the values of adc channel 6 and 7 are read and shown in log.
4. GPIOA_2 is adc channel 6.
5. GPIOA_3 is adc channel 7.
6. Input voltage 0-3.3V, Output data range 0-4095.

```
adc example! rtl8735b_evb/amebapro2

*** Booting Zephyr OS build 398525dea97b ***
ADC channel: 6, value: 4095
ADC channel: 7, value: 13
ADC channel: 6, value: 4095
ADC channel: 7, value: 14
ADC channel: 6, value: 4095
ADC channel: 7, value: 15
```

- NPU driver

1. Build the sample: platform/samples/npu/. The build process packages both the SCRFD and YOLOv4-tiny model binaries into the PT_NN_MDL partition of flash_zephyr.bin.
2. Use PG Tool to download flash_zephyr.bin.
3. Reset the EVB and enter the following command to list the supported NPU detection models: npu list
4. Use one of the following commands to run NPU inference:
   a. npu run face — Run the SCRFD face detection example. The sample loads scrfd.nb from flash and uses the pre-generated 576x320 RGB888 planar image converted from wls.jpg as the model input. After inference, it decodes the output tensors and prints the detected face count, confidence scores, and bounding-box coordinates. With the provided model and test image, 93 faces should be detected.
   b. npu run yolo — Run the YOLOv4-tiny object detection example. The sample loads yolov4_tiny.nb from flash and uses the pre-generated 416x416 RGB888 planar image converted from horses_416x416.jpg as the model input. After inference, it decodes the output tensors and prints each detected object's COCO class, confidence score, and bounding-box coordinates.
   c. npu run — Run face detection by default. This command is equivalent to npu run face.
5. Each command performs one inference run and prints the NPU inference time and total cycle count. After post-processing is completed, the sample automatically unloads the selected model and deinitializes the NPU. Only one NPU sample can run at a time.

```
uart:~$ npu list
face: SCRFD face detection, input 576x320, flash file scrfd.nb
yolo: YOLOv4-tiny object detection, input 416x416, flash file yolov4_tiny.nb
```

```
uart:~$ npu run face
NPU sample started: SCRFD face detection
[00:10:37.993,000] <inf> npu_sample: Ameba NPU sample on rtl8735b_evb/amebapro2
[00:10:37.993,000] <inf> npu_sample: Selected model: SCRFD face detection (scrfd.nb)
[00:10:37.993,000] <inf> npu_sample: NPU runtime version: 0x00020000
npu[70267260] vipdrv init, video memory heap base: 0x77000000, size: 0x01000000
npu[70267260] HASHMAP 0x0x7026c910(process-ID) INIT SUCCESS
npu[70267260] DATABASE 0x0x7026da1c(task-desc) INIT SUCCESS
npu[70267260] DATABASE 0x0x7026d9c8(memory-mgt) INIT SUCCESS
[00:10:38.173,000] <inf> npu_model_loader: Loaded scrfd.nb from NN_MDL flash partition: flash=0x08921800
HASHMAP 0x70286398(record-list) INIT SUCCESS
[00:10:38.201,000] <inf> npu_sample: Model loaded: inputs=1 outputs=9
[00:10:38.201,000] <inf> npu_sample: Use face test input: 576x320 RGB888 planar
[00:10:38.266,000] <inf> npu_sample: Inference 1/1 time: 19 us, total_cycle=9524120
[00:10:38.266,000] <inf> npu_sample: Output[0]: format=2 quant=2 size=5760
[00:10:38.267,000] <inf> npu_sample: Output[1]: format=2 quant=2 size=1440
[00:10:38.267,000] <inf> npu_sample: Output[2]: format=2 quant=2 size=360
[00:10:38.267,000] <inf> npu_sample: Output[3]: format=2 quant=2 size=23040
[00:10:38.267,000] <inf> npu_sample: Output[4]: format=2 quant=2 size=5760
[00:10:38.267,000] <inf> npu_sample: Output[5]: format=2 quant=2 size=1440
[00:10:38.268,000] <inf> npu_sample: Output[6]: format=2 quant=2 size=57600
[00:10:38.268,000] <inf> npu_sample: Output[7]: format=2 quant=2 size=14400
[00:10:38.268,000] <inf> npu_sample: Output[8]: format=2 quant=2 size=3600
[00:10:38.310,000] <inf> scrfd_postprocess: Detected 93 face(s)
[00:10:38.310,000] <inf> scrfd_postprocess: Face[0]: score=0.534 x1=556 y1=93 x2=563 y2=101 w=7 h=8
[00:10:38.310,000] <inf> scrfd_postprocess: Face[1]: score=0.513 x1=534 y1=110 x2=541 y2=119 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[2]: score=0.545 x1=29 y1=120 x2=36 y2=128 w=7 h=8
[00:10:38.310,000] <inf> scrfd_postprocess: Face[3]: score=0.534 x1=45 y1=116 x2=52 y2=124 w=7 h=8
[00:10:38.310,000] <inf> scrfd_postprocess: Face[4]: score=0.588 x1=86 y1=127 x2=93 y2=136 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[5]: score=0.556 x1=549 y1=124 x2=556 y2=133 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[6]: score=0.609 x1=446 y1=133 x2=453 y2=142 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[7]: score=0.652 x1=459 y1=133 x2=466 y2=142 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[8]: score=0.524 x1=60 y1=139 x2=68 y2=148 w=8 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[9]: score=0.566 x1=71 y1=143 x2=80 y2=153 w=9 h=10
[00:10:38.310,000] <inf> scrfd_postprocess: Face[10]: score=0.534 x1=490 y1=141 x2=497 y2=150 w=7 h=9
[00:10:38.310,000] <inf> scrfd_postprocess: Face[11]: score=0.577 x1=2 y1=144 x2=11 y2=157 w=9 h=13
[00:10:38.310,000] <inf> scrfd_postprocess: Face[12]: score=0.673 x1=214 y1=147 x2=221 y2=157 w=7 h=10
[00:10:38.310,000] <inf> scrfd_postprocess: Face[13]: score=0.556 x1=475 y1=145 x2=484 y2=155 w=9 h=10
[00:10:38.310,000] <inf> scrfd_postprocess: Face[14]: score=0.577 x1=501 y1=144 x2=509 y2=154 w=8 h=10
[00:10:38.310,000] <inf> scrfd_postprocess: Face[15]: score=0.566 x1=522 y1=149 x2=530 y2=159 w=8 h=10
```

```
uart:~$ npu run yolo
NPU sample started: YOLOv4-tiny object detection
[00:11:25.401,000] <inf> npu_sample: Ameba NPU sample on rtl8735b_evb/amebapro2
[00:11:25.401,000] <inf> npu_sample: Selected model: YOLOv4-tiny object detection (yolov4_tiny.nb)
[00:11:25.401,000] <inf> npu_sample: NPU runtime version: 0x00020000
npu[70267260] vipdrv init, video memory heap base: 0x77000000, size: 0x01000000
npu[70267260] HASHMAP 0x0x7026c910(process-ID) INIT SUCCESS
npu[70267260] DATABASE 0x0x7026da1c(task-desc) INIT SUCCESS
npu[70267260] DATABASE 0x0x7026d9c8(memory-mgt) INIT SUCCESS
[00:11:26.150,000] <inf> npu_model_loader: Loaded yolov4_tiny.nb from NN_MDL flash partition: flash=0x089b0000 size=
HASHMAP 0x7028aa1c(record-list) INIT SUCCESS
[00:11:26.311,000] <inf> npu_sample: Model loaded: inputs=1 outputs=2
[00:11:26.312,000] <inf> npu_sample: Use yolo test input: 416x416 RGB888 planar
[00:11:26.418,000] <inf> npu_sample: Inference 1/1 time: 53 us, total_cycle=26303056
[00:11:26.418,000] <inf> npu_sample: Output[0]: dims=13x13x255 format=2 quant=2 size=43095
[00:11:26.418,000] <inf> npu_sample: Output[1]: dims=26x26x255 format=2 quant=2 size=172380
[00:11:26.447,000] <inf> yolov4_tiny_postprocess: Detected 3 object(s)
[00:11:26.447,000] <inf> yolov4_tiny_postprocess: Object[0]: class=17 (horse) score=0.925 x1=0 y1=145 x2=204 y2=339
[00:11:26.447,000] <inf> yolov4_tiny_postprocess: Object[1]: class=17 (horse) score=0.739 x1=2 y1=145 x2=83 y2=217
[00:11:26.447,000] <inf> yolov4_tiny_postprocess: Object[2]: class=19 (cow) score=0.982 x1=233 y1=175 x2=325 y2=282
[00:11:26.471,000] <inf> npu_sample: YOLOv4-tiny object detection sample completed
```